

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

ServiceForge
(Business Capability Platform)
Configurable SaaS Service Modules

Version 1.0
April 14, 2026

Prepared by: Platform Architecture Team
Status: Draft

Version	Date	Author	Description
1.0	2026-04-14	Platform Architecture Team	Initial SRS release
1.1	2026-04-14	Pramitha Jayasooriya	Review and stakeholder feedback

Table of Contents

Table of Contents.....	2
1. Introduction	6
1.1 Purpose	6
1.2 Product Scope	6
1.3 Definitions and Acronyms.....	6
1.4 References	6
1.5 Document Overview	7
2. Overall Description.....	8
2.1 Product Perspective	8
2.2 Product Features (High Level)	8
2.3 User Classes and Characteristics	8
2.4 Operating Environment	9
2.5 Design and Implementation Constraints	9
2.6 Assumptions and Dependencies.....	9
3. System Architecture.....	10
3.1 Architecture Overview	10
3.2 Architecture Layers	10
3.2.1 API Gateway Layer.....	10
3.2.2 Core Service Module Layer	10
3.2.3 Configuration Engine Layer	10
3.2.4 Event Bus Layer	10
3.2.5 Shared Infrastructure Layer	10
3.3 Multi-Tenancy Model	10
3.4 Tenant Context Propagation	11
4. Functional Requirements.....	12
4.1 Tenant Management (FR-TM).....	12
FR-TM-001: Tenant Registration	12
FR-TM-002: API Key Management	12
FR-TM-003: Team Member Management.....	12
FR-TM-004: Plan and Billing Management.....	12
4.2 Service Module Catalog (FR-SC).....	12
FR-SC-001: Browse Service Catalog	12
FR-SC-002: Subscribe to Module.....	12
FR-SC-003: Unsubscribe from Module.....	12

4.3 Booking Module (FR-BK)	12
FR-BK-001: Slot Configuration	13
FR-BK-002: Create Booking	13
FR-BK-003: Check Availability	13
FR-BK-004: Cancel Booking	13
FR-BK-005: Booking Lifecycle Events	13
FR-BK-006: Overbooking Management	13
4.4 Payment Module (FR-PM)	13
FR-PM-001: Payment Gateway Integration	13
FR-PM-002: One-Time Payment	13
FR-PM-003: Subscription Billing	13
FR-PM-004: Refund Processing	14
FR-PM-005: Invoice Generation	14
FR-PM-006: Payment Events	14
4.5 Queue Management Module (FR-QM)	14
FR-QM-001: Queue Configuration	14
FR-QM-002: Enqueue	14
FR-QM-003: Dequeue and Serve	14
FR-QM-004: Queue Position and Wait Time	14
FR-QM-005: Queue Events	14
4.6 Notification Module (FR-NT)	15
FR-NT-001: Multi-Channel Delivery	15
FR-NT-002: Template Management	15
FR-NT-003: Delivery Tracking	15
4.7 Logging and Audit Module (FR-LG)	15
FR-LG-001: Structured Logging	15
FR-LG-002: Audit Trail	15
FR-LG-003: Log Search and Export	15
FR-LG-004: Alerting	15
4.8 Configuration Engine (FR-CE)	15
FR-CE-001: Schema Registration	15
FR-CE-002: Configuration CRUD	15
FR-CE-003: Configuration Hierarchy	16
FR-CE-004: Hot Reload	16
FR-CE-005: Configuration History	16
4.9 API Gateway (FR-GW)	16

FR-GW-001: Authentication.....	16
FR-GW-002: Rate Limiting	16
FR-GW-003: API Versioning.....	16
FR-GW-004: Request Validation	16
FR-GW-005: Usage Metering	16
4.10 Developer Tools (FR-DT).....	16
FR-DT-001: Auto-Generated API Documentation	17
FR-DT-002: Sandbox Environment	17
FR-DT-003: SDK Generation.....	17
FR-DT-004: Webhook Management.....	17
5. Use Cases	18
5.1 Tenant Onboarding Use Cases.....	18
5.2 Booking Module Use Cases	18
5.3 Payment Module Use Cases.....	20
5.4 Queue Management Use Cases	21
5.5 Configuration and Administration Use Cases.....	22
6. User Interface Specifications	24
6.1 Global Layout and Navigation	24
6.2 Screen Descriptions	24
6.2.1 Dashboard (Home)	24
6.2.2 Service Catalog	24
6.2.3 Module Configuration Screen	25
6.2.4 API Documentation (Interactive).....	25
6.2.5 API Key Management.....	25
6.2.6 Webhook Management.....	26
6.2.7 Billing and Usage.....	26
6.2.8 Logs and Monitoring	26
6.2.9 Team Management.....	27
6.3 UI Design Principles	27
7. Non-Functional Requirements	28
7.1 Performance.....	28
7.2 Scalability	28
7.3 Availability and Reliability.....	28
7.4 Security	28
7.5 Maintainability.....	29
7.6 Usability.....	29

8. External Interface Requirements	30
8.1 REST API Interface	30
8.2 Webhook Interface	30
8.3 Payment Gateway Interface	30
8.4 Notification Provider Interface	30
8.5 Management UI Interface	30
9. Constraints and Assumptions	31
9.1 Technical Constraints	31
9.2 Business Constraints	31
9.3 Assumptions	31
10. Appendices	32
10.1 Use Case Summary Matrix	32
10.2 Module-to-Event Mapping	32
10.3 Configuration Parameter Summary	32

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a comprehensive description of the Business Capability Platform (ServiceForge). It details the functional and non-functional requirements, use cases, user interface specifications, system architecture, and constraints for the platform. This document is intended for developers, architects, QA engineers, product managers, and stakeholders involved in the development and deployment of the ServiceForge.

1.2 Product Scope

The Business Capability Platform (ServiceForge) is a multi-tenant, configurable SaaS platform that provides pre-built business service modules such as Booking, Payment Processing, Queue Management, Notifications, and Logging/Audit. The target users are software developers and technical teams who need to integrate common business capabilities into their applications without building them from scratch.

Developers subscribe to service modules, configure them via a web-based Management UI or REST API, and integrate them into their applications through auto-generated API endpoints. The platform eliminates the need to code common business logic by providing parameter-driven configuration for all module behaviors.

1.3 Definitions and Acronyms

Term	Definition
ServiceForge	Business Capability Platform - the system described in this document
Tenant	A registered developer/organization with isolated configuration and data
Service Module	A self-contained business capability (e.g., Booking, Payment, Queue)
Configuration Schema	JSON Schema defining configurable parameters for a module
Management UI	Web-based portal for configuring and monitoring subscribed services
API Gateway	Entry point for all client API requests; handles auth, routing, rate limiting
Event Bus	Asynchronous messaging system (Kafka/RabbitMQ) connecting modules
RLS	Row-Level Security - PostgreSQL feature for multi-tenant data isolation
SLA	Service Level Agreement
SDK	Software Development Kit

1.4 References

- IEEE 830-1998 - Recommended Practice for Software Requirements Specifications
- OpenAPI Specification 3.1
- JSON Schema Draft 2020-12

- OAuth 2.0 Authorization Framework (RFC 6749)
- WSO2 API Manager Architecture Reference

1.5 Document Overview

Section 2 describes the overall product perspective and features. Section 3 details the system architecture. Section 4 covers all functional requirements organized by module. Section 5 documents use cases. Section 6 specifies the user interface designs. Section 7 lists non-functional requirements. Section 8 covers external interface requirements. Section 9 describes constraints and assumptions.

2. Overall Description

2.1 Product Perspective

ServiceForge is a standalone SaaS platform positioned between low-level infrastructure services (AWS, GCP) and fully custom application development. It is analogous to platforms like WSO2 API Manager (for API lifecycle management), Stripe (for payment abstraction), and Kafka UI (for event stream visualization), but unified into a single configurable platform with multiple business modules.

The platform operates as a cloud-hosted service or on prem. Tenants interact with ServiceForge through two channels: the Management UI (web portal) for configuration and monitoring, and the REST/GraphQL API for programmatic integration from their applications.

2.2 Product Features (High Level)

#	Feature	Description
F1	Multi-Tenant Service Platform	Isolated configuration and data per tenant with shared infrastructure
F2	Service Module Catalog	Browse, subscribe, and manage available business service modules
F3	Booking Module	Slot-based scheduling, availability management, reservation lifecycle
F4	Payment Module	Payment processing abstraction over Stripe/PayPal/Razorpay
F5	Queue Management Module	Virtual queuing with FIFO/priority disciplines and notifications
F6	Logging and Audit Module	Structured logging, audit trails, alerting, and analytics
F7	Notification Module	Email, SMS, push notification delivery with templates
F8	Configuration Engine	JSON Schema-driven parameter management with UI auto-generation
F9	API Gateway	Authentication, rate limiting, routing, versioning, and metering
F10	Management UI Portal	Web dashboard for configuration, monitoring, API docs, and billing
F11	Event Bus Integration	Asynchronous inter-module communication and webhook delivery
F12	Developer Tools	Auto-generated API docs, SDKs, sandbox mode, and CLI tools

2.3 User Classes and Characteristics

User Class	Access Level	Description
Platform Admin	Full system access	Manages platform operations, tenant provisioning, global config, monitoring

Tenant Developer	Tenant-scoped access	Subscribes to modules, configures via UI/API, integrates with their app
Tenant Admin	Tenant-scoped admin	Manages team members, billing, API keys, and usage policies within tenant
End User (indirect)	No direct access	The tenant developer's customer who interacts via the tenant's app

2.4 Operating Environment

- Cloud deployment on Kubernetes (AWS EKS / GCP GKE / Azure AKS)
- PostgreSQL 15+ with Row-Level Security for multi-tenant data isolation
- Apache Kafka 3.x or RabbitMQ 3.12+ for event messaging
- Redis 7.x for caching and session management
- Management UI: Modern browsers (Chrome 100+, Firefox 100+, Safari 16+, Edge 100+)
- API clients: Any HTTP client supporting REST/JSON or GraphQL

2.5 Design and Implementation Constraints

- All service modules must support multi-tenancy with strict data isolation
- Configuration changes must be applied without service restart (hot reload)
- API response latency must not exceed 200ms at p95 under normal load
- All inter-module communication must be asynchronous via the event bus
- Platform must comply with GDPR and SOC 2 Type II requirements
- All APIs must follow OpenAPI 3.1 specification

2.6 Assumptions and Dependencies

- Tenants provide their own payment gateway API keys (Stripe, PayPal, etc.)
- Tenants handle end-user authentication in their own applications
- SMS/email delivery depends on third-party providers (Twilio, SendGrid)
- Cloud infrastructure is pre-provisioned and managed separately
- Tenants have basic REST API integration knowledge

3. System Architecture

3.1 Architecture Overview

The ServiceForge follows a layered microservices architecture with five distinct tiers: Client Layer, API Gateway Layer, Core Service Module Layer, Configuration Engine Layer, and Shared Infrastructure Layer. Each service module is deployed as an independent microservice with its own database schema, API surface, and event publishers.

3.2 Architecture Layers

3.2.1 API Gateway Layer

The API Gateway is the single entry point for all client requests. It is responsible for tenant authentication (via API keys, OAuth 2.0, or JWT), request routing to the appropriate service module, rate limiting per tenant and per endpoint, API version management, request/response validation and transformation, and usage metering for billing.

Technology: Kong Gateway or custom implementation using Node.js (Fastify) with a reverse proxy.

3.2.2 Core Service Module Layer

Each service module is a self-contained microservice that provides a specific business capability. Modules are independently deployable, scalable, and configurable. They communicate with each other exclusively through the event bus. Each module registers a configuration schema that defines its tunable parameters.

3.2.3 Configuration Engine Layer

The Configuration Engine stores and serves per-tenant settings for every module. It validates configurations against registered JSON Schemas, supports a configuration hierarchy (global defaults, plan-level overrides, tenant-specific overrides), and emits config.updated events for hot-reload by service modules.

3.2.4 Event Bus Layer

All inter-module communication flows through Apache Kafka or RabbitMQ. Events follow a standardized envelope format including tenant_id, event_type, timestamp, correlation_id, and payload. The event bus also powers webhook delivery to tenant endpoints.

3.2.5 Shared Infrastructure Layer

This layer includes PostgreSQL (multi-tenant database with RLS), Redis (caching, rate limiting, session store), S3/MinIO (object storage for files and exports), and the observability stack (Prometheus, Grafana, ELK for metrics, tracing, and logging).

3.3 Multi-Tenancy Model

ServiceForge uses a shared-database, row-level isolation model as the default. Every table includes a tenant_id column, enforced by PostgreSQL Row-Level Security (RLS) policies. This ensures that a tenant can never access another tenant's data, even through direct SQL. For enterprise clients requiring maximum isolation, a dedicated database per tenant option is available as a premium tier.

3.4 Tenant Context Propagation

Every incoming API request is authenticated at the gateway, which injects a tenant context (tenant_id, plan_id, permissions) into the request headers. This context propagates through every service call, database query, cache key prefix, and event message. No service module operates without an explicit tenant context.

4. Functional Requirements

4.1 Tenant Management (FR-TM)

FR-TM-001: Tenant Registration

The system shall allow a new developer to register by providing organization name, email, and password. Upon registration, the system shall create a new tenant, provision isolated database resources, generate a default API key, and assign the Free plan.

FR-TM-002: API Key Management

The system shall allow tenant admins to create, rotate, revoke, and list API keys. Each API key shall be scoped to specific modules and environments (sandbox/production). The system shall support a maximum of 20 active API keys per tenant.

FR-TM-003: Team Member Management

The system shall allow tenant admins to invite team members via email, assign roles (Admin, Developer, Viewer), and revoke access. Role-based access control (RBAC) shall restrict Management UI and API access based on assigned roles.

FR-TM-004: Plan and Billing Management

The system shall support multiple subscription plans (Free, Starter, Pro, Enterprise) with configurable limits per module (e.g., max bookings/day, API calls/month). Plan upgrades and downgrades shall take effect immediately with prorated billing.

4.2 Service Module Catalog (FR-SC)

FR-SC-001: Browse Service Catalog

The system shall display all available service modules with name, description, feature list, pricing tier, and configuration options. The catalog shall be filterable by category (scheduling, payment, communication, monitoring).

FR-SC-002: Subscribe to Module

The system shall allow tenants to subscribe to a service module with a single click. Upon subscription, the system shall provision necessary database tables, register event subscriptions, activate API routes, generate module-specific API documentation, and apply default configuration from the tenant's plan.

FR-SC-003: Unsubscribe from Module

The system shall allow tenants to unsubscribe from a module. The system shall deactivate API routes immediately, retain data for 30 days (configurable), and emit a module.unsubscribed event.

4.3 Booking Module (FR-BK)

FR-BK-001: Slot Configuration

The system shall allow tenants to configure booking slots including slot duration (5-480 minutes), working hours per day of week, buffer time between bookings, blackout dates and recurring holidays, maximum concurrent bookings per slot, and advance booking window (1-365 days).

FR-BK-002: Create Booking

The system shall accept booking creation requests containing customer identifier, requested slot (date/time), optional metadata (name, email, notes). The system shall validate slot availability, enforce tenant-configured limits, and return a booking confirmation with a unique booking ID.

FR-BK-003: Check Availability

The system shall provide an endpoint that returns available slots for a given date range, considering existing bookings, buffer times, blackout dates, working hours, and overbooking percentage if configured.

FR-BK-004: Cancel Booking

The system shall allow booking cancellation. The system shall enforce the tenant-configured cancellation policy (e.g., free cancellation up to X hours before), calculate refund percentage, emit booking.cancelled event, and release the slot for rebooking.

FR-BK-005: Booking Lifecycle Events

The system shall emit events for all booking state transitions: booking.created, booking.confirmed, booking.reminded (configurable hours before), booking.completed, booking.cancelled, and booking.no_show.

FR-BK-006: Overbooking Management

The system shall support configurable overbooking percentage (0-50%). When overbooking is enabled, the system shall track actual capacity vs booked capacity and emit booking.overbooked_warning when threshold is reached.

4.4 Payment Module (FR-PM)**FR-PM-001: Payment Gateway Integration**

The system shall support integration with Stripe, PayPal, and Razorpay. Tenants shall configure their own gateway API keys through the Management UI. The system shall abstract gateway-specific APIs behind a unified payment interface.

FR-PM-002: One-Time Payment

The system shall process one-time charges with amount, currency, customer identifier, and optional metadata. The system shall return a payment ID, status (pending/succeeded/failed), and gateway transaction reference.

FR-PM-003: Subscription Billing

The system shall support recurring billing with configurable intervals (weekly, monthly, annually), trial periods, proration, and automatic retry on failure. Tenants configure subscription plans through the Management UI.

FR-PM-004: Refund Processing

The system shall process full and partial refunds. Refunds shall be linked to the original payment, respect the tenant-configured auto-refund rules, and emit payment.refunded events.

FR-PM-005: Invoice Generation

The system shall auto-generate invoices for all payments with configurable invoice templates. Invoices shall include line items, tax calculations (based on tenant-configured rules), and PDF export.

FR-PM-006: Payment Events

The system shall emit: payment.initiated, payment.succeeded, payment.failed, payment.refunded, subscription.created, subscription.cancelled, subscription.renewed, and invoice.generated.

4.5 Queue Management Module (FR-QM)

FR-QM-001: Queue Configuration

The system shall allow tenants to configure: queue discipline (FIFO, priority-based, weighted fair), maximum queue size, estimated service time per item, VIP/priority tier definitions, auto-expiry timeout, and operating hours.

FR-QM-002: Enqueue

The system shall accept enqueue requests with customer identifier, optional priority level, and metadata. The system shall return a queue position, estimated wait time, and unique queue ticket ID.

FR-QM-003: Dequeue and Serve

The system shall support manual dequeue (via API call) and auto-dequeue (timer-based). When an item is dequeued, the system shall notify the customer, update positions for remaining items, and emit queue.item_served event.

FR-QM-004: Queue Position and Wait Time

The system shall provide real-time queue position and estimated wait time via API endpoint and webhook. Wait time calculation shall consider the queue discipline, average service time, and items ahead in queue.

FR-QM-005: Queue Events

The system shall emit: queue.item_added, queue.item_served, queue.item_expired, queue.item_cancelled, queue.capacity_warning, and queue.position_updated.

4.6 Notification Module (FR-NT)

FR-NT-001: Multi-Channel Delivery

The system shall support notification delivery via email (SendGrid/SES), SMS (Twilio), push notifications (FCM/APNs), and webhook callbacks. Tenants configure preferred channels per event type.

FR-NT-002: Template Management

The system shall provide a template engine with variable substitution (e.g., {{customer_name}}, {{booking_time}}). Templates shall be configurable per tenant, per channel, and per event type through the Management UI.

FR-NT-003: Delivery Tracking

The system shall track delivery status (queued, sent, delivered, failed, bounced) for every notification. Delivery reports shall be accessible via API and the Management UI dashboard.

4.7 Logging and Audit Module (FR-LG)

FR-LG-001: Structured Logging

The system shall capture structured logs for all API calls, configuration changes, module events, and system errors. Each log entry shall include timestamp, tenant_id, module, action, actor, IP address, request/response summary, and correlation_id.

FR-LG-002: Audit Trail

The system shall maintain an immutable audit trail for all state-changing operations. Audit records shall be tamper-evident and retained for the tenant-configured period (default: 90 days, max: 7 years).

FR-LG-003: Log Search and Export

The system shall provide full-text search across logs with filtering by date range, module, severity, and action type. Logs shall be exportable in JSON and CSV formats.

FR-LG-004: Alerting

The system shall support configurable alert rules (e.g., 'notify if payment failure rate > 5% in 1 hour'). Alerts shall trigger notifications via the Notification Module.

4.8 Configuration Engine (FR-CE)

FR-CE-001: Schema Registration

Each service module shall register a JSON Schema document defining all configurable parameters, including data types, defaults, validation rules, min/max values, enumerations, and UI rendering hints (slider, dropdown, toggle, text input, nested form).

FR-CE-002: Configuration CRUD

The system shall provide API endpoints to read, create, update, and reset configurations per tenant per module. All changes shall be validated against the registered schema before persistence.

FR-CE-003: Configuration Hierarchy

The system shall resolve configuration values in order: tenant-specific override, then plan-level default, then global default. Tenants shall only be able to override parameters permitted by their plan.

FR-CE-004: Hot Reload

When a configuration is updated, the system shall emit a config.updated event. Service modules shall subscribe to this event and apply changes within 5 seconds without restart.

FR-CE-005: Configuration History

The system shall maintain a versioned history of all configuration changes per tenant, including who changed what, when, and the previous value. Rollback to any previous version shall be supported.

4.9 API Gateway (FR-GW)

FR-GW-001: Authentication

The gateway shall support API key authentication (header-based), OAuth 2.0 client credentials flow, and JWT bearer token validation. All endpoints except /health and /docs shall require authentication.

FR-GW-002: Rate Limiting

The gateway shall enforce rate limits per tenant per endpoint, configurable by plan (e.g., Free: 100 req/min, Pro: 10,000 req/min). Rate limit status shall be returned in response headers (X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset).

FR-GW-003: API Versioning

The gateway shall support URL-based versioning (/api/v1/, /api/v2/). Deprecated versions shall return Sunset and Deprecation headers with migration guidance.

FR-GW-004: Request Validation

The gateway shall validate all incoming requests against OpenAPI schemas before routing to service modules. Invalid requests shall be rejected with structured error responses (RFC 7807 Problem Details).

FR-GW-005: Usage Metering

The gateway shall track API call counts per tenant, per module, per endpoint, per time period. Metering data shall feed into the billing system and be visible in the Management UI dashboard.

4.10 Developer Tools (FR-DT)

FR-DT-001: Auto-Generated API Documentation

The system shall generate OpenAPI 3.1 specification documents per tenant, including only the modules they have subscribed to. Interactive documentation (Swagger UI) shall be accessible from the Management UI.

FR-DT-002: Sandbox Environment

Each tenant shall have a sandbox environment with test data, simulated payment processing, and no rate limits. Sandbox API keys shall be distinct from production keys.

FR-DT-003: SDK Generation

The system shall provide auto-generated client SDKs in JavaScript/TypeScript, Python, Java, Go, and PHP. SDKs shall be downloadable from the Management UI and published to package registries.

FR-DT-004: Webhook Management

The system shall allow tenants to register webhook URLs for specific event types, configure retry policies (max 5 retries with exponential backoff), view delivery logs, and test webhooks with sample payloads.

5. Use Cases

5.1 Tenant Onboarding Use Cases

Use Case ID	UC-01
Name	Developer Registration and Project Setup
Actor(s)	Tenant Developer
Priority	High
Precondition	Developer has a valid email address and internet access
Main Flow	1. Developer navigates to ServiceForge registration page 2. Developer enters organization name, email, password 3. System validates input and creates tenant account 4. System provisions isolated database resources (schema/RLS policies) 5. System generates default API key pair (sandbox + production) 6. System assigns Free plan with default module limits 7. System redirects developer to Management UI dashboard 8. Dashboard displays onboarding wizard with getting-started guide
Alt/Exception	A1: Email already registered - system prompts login or password reset A2: Provisioning failure - system retries, notifies admin, shows error to user
Postcondition	Tenant account is active with sandbox API key ready for use

Use Case ID	UC-02
Name	Subscribe to Service Module
Actor(s)	Tenant Developer, Tenant Admin
Priority	High
Precondition	Tenant is authenticated and has an active account
Main Flow	9. Developer opens Service Catalog in Management UI 10. Developer browses modules filtered by category 11. Developer selects a module (e.g., Booking) and reviews details 12. Developer clicks 'Subscribe' button 13. System checks plan limits and available module slots 14. System provisions module resources (tables, routes, events) 15. System applies default configuration from tenant's plan 16. System generates module-specific API documentation 17. Module appears in tenant's dashboard as 'Active'
Alt/Exception	A1: Plan limit reached - system prompts upgrade A2: Module requires higher plan - system shows upgrade options
Postcondition	Module is active, API endpoints are live, default config applied

5.2 Booking Module Use Cases

Use Case ID	UC-03
Name	Configure Booking Parameters
Actor(s)	Tenant Developer
Priority	High
Precondition	Tenant has subscribed to the Booking module
Main Flow	18. Developer opens Booking module configuration in Management UI 19. System displays auto-generated form based on Booking JSON Schema 20. Developer sets slot duration to 60 minutes 21. Developer sets max bookings per day to 200 22. Developer configures working hours (Mon-Fri, 9AM-5PM) 23. Developer sets cancellation policy to free cancel up to 24 hours before 24. Developer clicks 'Save Configuration' 25. System validates against schema and saves configuration 26. System emits config.updated event 27. Booking module picks up new config within 5 seconds
Alt/Exception	A1: Validation error - system highlights invalid fields with error messages A2: Parameter exceeds plan limit - system shows plan restriction
Postcondition	Booking module operates with new configuration

Use Case ID	UC-04
Name	End User Creates a Booking (via Tenant App)
Actor(s)	End User (via Tenant Developer's application)
Priority	High
Precondition	Tenant app is integrated with ServiceForge Booking API
Main Flow	28. End user selects date/time in tenant's booking interface 29. Tenant app calls POST /api/v1/bookings with slot details 30. API Gateway authenticates request via API key 31. Gateway routes to Booking module 32. Booking module checks slot availability against tenant config 33. Booking module creates reservation and assigns booking ID 34. System emits booking.created event 35. Payment module (if subscribed) listens and initiates charge 36. Notification module (if subscribed) sends confirmation 37. API returns booking confirmation to tenant app
Alt/Exception	A1: Slot unavailable - returns 409 Conflict with next available slots A2: Max bookings reached - returns 429 with limit details A3: Payment fails - booking marked as pending_payment, retry event emitted
Postcondition	Booking is created, slot capacity reduced, events dispatched

Use Case ID	UC-05
--------------------	-------

Name	Cancel Booking with Refund
Actor(s)	End User (via Tenant Developer's application)
Priority	High
Precondition	A confirmed booking exists
Main Flow	38. End user requests cancellation via tenant's app 39. Tenant app calls PUT /api/v1/bookings/{id}/cancel 40. Booking module checks cancellation policy (hours before start) 41. Booking module calculates refund percentage 42. System emits booking.cancelled event with refund info 43. Payment module processes refund based on calculated amount 44. Notification module sends cancellation confirmation 45. Slot is released and availability updated
Alt/Exception	A1: Outside cancellation window - return partial refund or no refund per policy A2: Refund processing fails - system queues retry and notifies tenant admin
Postcondition	Booking cancelled, refund processed, slot released

5.3 Payment Module Use Cases

Use Case ID	UC-06
Name	Configure Payment Gateway
Actor(s)	Tenant Developer
Priority	High
Precondition	Tenant has subscribed to Payment module
Main Flow	46. Developer opens Payment configuration in Management UI 47. Developer selects payment gateway (Stripe/PayPal/Razorpay) 48. Developer enters their gateway API keys (stored encrypted) 49. Developer configures currency, tax rules, and retry policy 50. Developer sets up webhook URL for payment events 51. System validates gateway connectivity with a test ping 52. System saves configuration and activates payment endpoints
Alt/Exception	A1: Invalid API key - system shows gateway validation error A2: Gateway timeout on test - system warns but allows save
Postcondition	Payment module connected to tenant's gateway, endpoints active

Use Case ID	UC-07
Name	Process One-Time Payment
Actor(s)	End User (via Tenant Developer's application)
Priority	High

Precondition	Payment module configured with valid gateway credentials
Main Flow	53. Tenant app calls POST /api/v1/payments/charge with amount, currency, customer_id 54. Payment module resolves tenant's gateway configuration 55. Payment module translates request to gateway-specific API call 56. Gateway processes charge and returns result 57. System stores payment record with gateway transaction reference 58. System emits payment.succeeded or payment.failed event 59. Notification module sends receipt if configured 60. API returns payment status to tenant app
Alt/Exception	A1: Card declined - return payment.failed with decline reason A2: Gateway unavailable - queue for retry, return 503 A3: Duplicate charge detected - return idempotency conflict
Postcondition	Payment processed, record stored, events dispatched

5.4 Queue Management Use Cases

Use Case ID	UC-08
Name	Enqueue Customer
Actor(s)	End User (via Tenant Developer's application)
Priority	Medium
Precondition	Queue module subscribed and configured
Main Flow	61. Tenant app calls POST /api/v1/queue/enqueue with customer_id and priority 62. Queue module validates against max queue size 63. System assigns queue position based on discipline (FIFO/priority) 64. System calculates estimated wait time 65. System emits queue.item_added event 66. API returns queue ticket ID, position, and estimated wait
Alt/Exception	A1: Queue full - return 429 with estimated next opening A2: Duplicate enqueue - return existing ticket info
Postcondition	Customer is in queue with assigned position

Use Case ID	UC-09
Name	Serve Next in Queue
Actor(s)	Tenant Developer's application (automated or manual trigger)
Priority	Medium
Precondition	At least one item in the queue
Main Flow	67. Tenant app calls POST /api/v1/queue/dequeue 68. Queue module retrieves next item based on discipline

	69. System updates status to 'serving' 70. System recalculates positions for remaining items 71. System emits queue.item_served event 72. Notification module notifies the customer 73. API returns served item details
Alt/Exception	A1: Queue empty - return 204 No Content A2: Customer expired (no-show) - skip and dequeue next
Postcondition	Item dequeued, remaining positions updated, customer notified

5.5 Configuration and Administration Use Cases

Use Case ID	UC-10
Name	View Configuration History and Rollback
Actor(s)	Tenant Admin
Priority	Medium
Precondition	At least one configuration change has been made
Main Flow	74. Admin opens module configuration page 75. Admin clicks 'Configuration History' tab 76. System displays versioned list of changes with timestamps and authors 77. Admin selects a previous version to compare 78. System shows diff between current and selected version 79. Admin clicks 'Rollback to This Version' 80. System validates the old configuration against current schema 81. System applies rollback and emits config.updated event
Alt/Exception	A1: Old config incompatible with current schema - system shows migration hints A2: Rollback fails validation - system shows errors, no change applied
Postcondition	Configuration reverted to selected version

Use Case ID	UC-11
Name	Monitor Platform Health via Dashboard
Actor(s)	Tenant Developer, Tenant Admin
Priority	Medium
Precondition	Tenant has at least one active module
Main Flow	82. User logs into Management UI 83. Dashboard loads with real-time metrics for subscribed modules 84. User views booking rates, payment success rates, queue lengths 85. User views API call volume chart and error rate 86. User drills into specific module for detailed metrics 87. User configures alert thresholds for specific metrics 88. System saves alert rules and begins monitoring

Alt/Exception	A1: Metrics service unavailable - dashboard shows stale data with timestamp A2: No data yet - dashboard shows onboarding prompts
Postcondition	User has visibility into platform health and active alerts

Use Case ID	UC-12
Name	Manage Webhook Endpoints
Actor(s)	Tenant Developer
Priority	High
Precondition	Tenant has active subscribed modules
Main Flow	89. Developer opens Webhooks section in Management UI 90. Developer clicks 'Add Webhook Endpoint' 91. Developer enters URL, selects events to subscribe (e.g., booking.created, payment.succeeded) 92. Developer sets retry policy and secret for signature verification 93. System validates URL is reachable 94. Developer clicks 'Test' to send sample payload 95. System sends test webhook and displays response status 96. Developer saves webhook configuration
Alt/Exception	A1: URL unreachable - system warns but allows save A2: Test payload fails - system shows response details for debugging
Postcondition	Webhook endpoint registered and receiving events

6. User Interface Specifications

This section describes the key screens and UI components of the Management UI portal. The UI follows a responsive design pattern, is accessible (WCAG 2.1 AA compliant), and uses a consistent design system across all modules.

6.1 Global Layout and Navigation

The Management UI uses a left sidebar navigation with collapsible sections. The top bar displays the current tenant name, environment switcher (Sandbox/Production), notification bell, and user avatar with dropdown. The sidebar contains: Dashboard, Service Catalog, Subscribed Modules (expandable with each active module), Configuration, API Keys, Webhooks, Team Members, Billing, Logs, and Settings.

6.2 Screen Descriptions

6.2.1 Dashboard (Home)

The primary landing page after login. Provides an at-a-glance overview of platform health and module activity.

Layout:

- Top row: 4 metric cards in a horizontal grid - Total API Calls (24h), Active Bookings, Payment Volume (24h), Queue Length
- Each metric card shows the current value, percentage change from previous period, and a sparkline mini-chart
- Middle section: Full-width time-series chart showing API request volume over last 7 days, with toggles per module
- Bottom left: Recent Activity feed (real-time log of events like bookings, payments, config changes)
- Bottom right: System Health panel showing module status indicators (green/yellow/red) and uptime percentages
- If no modules are subscribed: Show onboarding wizard with 'Get Started' card linking to Service Catalog

6.2.2 Service Catalog

Browse and subscribe to available business service modules.

Layout:

- Top: Search bar with category filter chips (All, Scheduling, Payments, Communication, Monitoring, Analytics)
- Grid of module cards (2-3 per row depending on viewport), each card showing: Module icon, name, short description, feature highlights (3-4 bullet points), pricing tier badge (Free/Starter/Pro/Enterprise), and 'Subscribe' or 'Subscribed' button
- Clicking a card opens a detail drawer/modal with: Full description, configuration parameter preview, API endpoint summary, pricing breakdown, and 'Subscribe Now' CTA

- Subscribed modules show a green checkmark badge and link to their configuration page

6.2.3 Module Configuration Screen

The central screen for customizing module behavior. This is the key differentiator of the platform.

Layout:

- Left panel (30% width): Navigation tree of configuration sections (e.g., for Booking: General, Schedule, Slots, Cancellation Policy, Overbooking, Notifications)
- Right panel (70% width): Auto-generated form based on the module's JSON Schema. Form controls are determined by schema `ui_component` hints: sliders for numeric ranges, toggles for booleans, dropdowns for enums, date/time pickers for schedules, nested collapsible sections for complex objects
- Each field shows: Label, current value, description tooltip, validation state, and 'Reset to Default' icon
- Top bar of config panel: 'Save' button (primary), 'Reset All to Defaults' (secondary), 'View as JSON' toggle, 'Configuration History' link
- After saving: Green success toast notification, and a subtle diff indicator showing what changed
- JSON view: Side-by-side editor showing raw JSON config with syntax highlighting and validation

6.2.4 API Documentation (Interactive)

Auto-generated, tenant-specific API documentation with a Swagger UI-like interactive interface.

Layout:

- Left sidebar: Expandable tree of API endpoints grouped by module (e.g., Booking > POST /bookings, GET /bookings/availability)
- Main panel per endpoint: HTTP method badge, URL path, description, request parameters table, request body schema with example JSON, response schemas for each status code, and 'Try It' interactive section
- 'Try It' section: Pre-filled with sandbox API key, editable request body, 'Send Request' button, and response viewer with status code, headers, and formatted JSON body
- Top bar: Environment switcher (Sandbox/Production), language selector for code examples (cURL, JavaScript, Python, Java, Go, PHP), and download buttons for OpenAPI spec and SDK

6.2.5 API Key Management

Create, manage, and monitor API keys.

Layout:

- Table listing all API keys: Key name, masked key value (click to reveal), environment (Sandbox/Production), module scope, created date, last used date, status (Active/Revoked)
- Actions column: Copy key, Rotate (generates new key, old key expires in 24h), Revoke (immediate)

- 'Create New Key' button opens modal: Key name, environment selection, module scope checkboxes, optional IP whitelist
- Usage metrics per key: API calls in last 24h, 7d, 30d with sparkline

6.2.6 Webhook Management

Register and monitor webhook endpoints for event delivery.

Layout:

- Table listing registered webhooks: URL (truncated with tooltip), subscribed events (pills/tags), status (Active/Paused/Failing), success rate (%), last delivery timestamp
- 'Add Webhook' button opens configuration form: URL input, event type multi-select (grouped by module), secret key (auto-generated, copyable), retry policy selector (None, 3x, 5x with exponential backoff)
- Delivery log drawer: Clicking a webhook row expands to show recent delivery attempts with timestamp, HTTP status code, response time, request payload preview, response body preview, and manual retry button
- Test button: Sends sample payload for selected event type and shows real-time response

6.2.7 Billing and Usage

View usage metrics, manage subscription plan, and access invoices.

Layout:

- Top: Current plan card showing plan name, price, renewal date, and 'Upgrade Plan' button
- Usage meters (horizontal progress bars): API Calls used/limit, Bookings used/limit, Payments processed/limit, Storage used/limit. Each shows current period usage with color coding (green < 70%, yellow 70-90%, red > 90%)
- Usage trend chart: Line chart showing daily API call volume for current billing period
- Invoice history table: Date, invoice number, amount, status (Paid/Pending/Failed), download PDF button
- Payment method section: Current card on file (masked), 'Update Payment Method' button

6.2.8 Logs and Monitoring

Search, filter, and analyze platform logs and audit trails.

Layout:

- Top: Search bar with full-text search, filter chips for module, severity (Info/Warning/Error), date range picker
- Main content: Scrollable log table with timestamp, module, action, severity badge (color-coded), summary, and actor
- Clicking a log entry expands inline to show full request/response details, correlation ID, tenant context, and execution duration
- Right sidebar (collapsible): Quick filters for common queries, saved filter presets, and export buttons (JSON, CSV)

- Real-time toggle: When enabled, new logs stream in at the top with subtle highlight animation

6.2.9 Team Management

Invite and manage team members with role-based access control.

Layout:

- Table listing team members: Name, email, role (Admin/Developer/Viewer), status (Active/Pending/Deactivated), last login date
- 'Invite Member' button: Email input, role selector dropdown, optional module access restrictions
- Role permissions matrix panel: Shows which actions each role can perform across modules (View Config, Edit Config, View Logs, Manage Keys, Manage Billing)
- Pending invitations section: Shows outstanding invites with resend and revoke options

6.3 UI Design Principles

- Consistent design system: Shared component library (buttons, forms, cards, tables, modals, toasts) across all screens
- Progressive disclosure: Show essential information first, details on demand (expand, drawer, modal)
- Immediate feedback: All actions produce visible feedback within 200ms (loading states, success/error toasts)
- Responsive layout: Functional on desktop (1024px+), tablet (768px+), and mobile (320px+)
- Keyboard accessible: All interactive elements reachable via Tab, actions via Enter/Space
- Dark mode support: System-preference-aware with manual toggle
- Empty states: Every list/table has a meaningful empty state with action prompt (e.g., 'No bookings yet. Configure your first booking module.')
- Error states: Form validation inline at field level, API errors displayed in contextual toast/banner

7. Non-Functional Requirements

7.1 Performance

ID	Requirement	Target
NFR-P1	API response latency (p50)	< 50ms
NFR-P2	API response latency (p95)	< 200ms
NFR-P3	API response latency (p99)	< 500ms
NFR-P4	Management UI page load time	< 2 seconds
NFR-P5	Configuration hot reload propagation	< 5 seconds
NFR-P6	Event bus message delivery latency	< 100ms (p95)
NFR-P7	Webhook delivery latency	< 2 seconds from event
NFR-P8	Database query execution time	< 50ms (p95)

7.2 Scalability

ID	Requirement	Target
NFR-S1	Concurrent tenants	10,000+
NFR-S2	API requests per second (platform total)	50,000 RPS
NFR-S3	Events processed per second	100,000 EPS
NFR-S4	Horizontal scaling	All services auto-scale on CPU/memory thresholds
NFR-S5	Database connections per module	Connection pooling with max 100 per pod

7.3 Availability and Reliability

ID	Requirement	Target
NFR-A1	Platform uptime SLA	99.95% (excludes planned maintenance)
NFR-A2	Recovery Time Objective (RTO)	< 15 minutes
NFR-A3	Recovery Point Objective (RPO)	< 1 minute
NFR-A4	Zero-downtime deployments	Required for all services
NFR-A5	Multi-AZ deployment	Active-active across 2+ availability zones

7.4 Security

ID	Requirement
----	-------------

NFR-SEC1	All API communication encrypted via TLS 1.3
NFR-SEC2	Tenant data isolation enforced at database level (RLS)
NFR-SEC3	API keys hashed (bcrypt) at rest, gateway credentials encrypted (AES-256)
NFR-SEC4	OWASP Top 10 vulnerabilities addressed in all components
NFR-SEC5	Audit log tamper protection (append-only, hash-chained)
NFR-SEC6	Rate limiting to prevent brute force and DDoS
NFR-SEC7	RBAC enforced on all Management UI and API actions
NFR-SEC8	SOC 2 Type II compliance
NFR-SEC9	GDPR compliance: data export, deletion, consent management
NFR-SEC10	Annual penetration testing by third-party auditor

7.5 Maintainability

- All services containerized (Docker) with Helm charts for Kubernetes deployment
- Infrastructure as Code (Terraform) for all cloud resources
- CI/CD pipeline with automated testing (unit > 80% coverage, integration, end-to-end)
- Centralized logging and distributed tracing (OpenTelemetry)
- API backward compatibility maintained for minimum 12 months after deprecation notice

7.6 Usability

- New developer should complete registration to first API call in under 15 minutes
- Module subscription and basic configuration achievable without reading documentation
- Management UI follows WCAG 2.1 AA accessibility guidelines
- In-context help tooltips on all configuration fields
- Interactive API documentation with working 'Try It' functionality

8. External Interface Requirements

8.1 REST API Interface

All service modules expose RESTful APIs following OpenAPI 3.1 specification. Base URL pattern: `https://api.ServiceForge.io/v{version}/{module}/`. Request/response format: JSON (`application/json`). Authentication: API key in `X-API-Key` header or OAuth 2.0 Bearer token. Error responses follow RFC 7807 Problem Details format with type, title, status, detail, and instance fields.

8.2 Webhook Interface

The platform delivers events to tenant-registered HTTP endpoints via POST requests. Webhook payload includes event metadata (id, type, timestamp, tenant_id) and module-specific data. Request includes HMAC-SHA256 signature in `X-ServiceForge-Signature` header for verification. Retry policy: exponential backoff (1s, 5s, 25s, 125s, 625s) up to 5 attempts.

8.3 Payment Gateway Interface

The Payment module integrates with external payment gateways via their official server-side SDKs. Supported gateways: Stripe (via `stripe-node`), PayPal (via `@paypal/checkout-server-sdk`), Razorpay (via `razorpay`). Gateway credentials are stored encrypted and never exposed in API responses or logs.

8.4 Notification Provider Interface

The Notification module integrates with: SendGrid and Amazon SES (email), Twilio (SMS), Firebase Cloud Messaging and Apple Push Notification service (push). Tenants provide their own provider API keys. The platform abstracts provider-specific APIs behind a unified notification interface.

8.5 Management UI Interface

Single-page application (SPA) built with React and TypeScript. Communicates with backend via REST API (same as tenant API, with Management scope). Real-time updates via WebSocket connection for live metrics and log streaming. Responsive design supporting desktop (1024px+), tablet (768px+), and mobile (320px+) viewports.

9. Constraints and Assumptions

9.1 Technical Constraints

- PostgreSQL 15+ required for RLS JSON path features
- Kubernetes 1.28+ required for deployment orchestration
- Maximum API request payload size: 10MB
- Maximum webhook payload size: 1MB
- Configuration schema maximum depth: 5 levels of nesting
- Maximum tenant team size: 100 members (configurable per plan)

9.2 Business Constraints

- Free plan limited to 2 modules, 1000 API calls/day, sandbox only
- Starter plan: 5 modules, 50,000 API calls/day, production access
- Pro plan: All modules, 500,000 API calls/day, priority support
- Enterprise plan: Dedicated resources, custom SLA, dedicated database option

9.3 Assumptions

- Cloud infrastructure (AWS/GCP/Azure) is provisioned and managed by the platform operations team
- Tenant developers have basic REST API knowledge and can integrate HTTP endpoints
- Third-party services (Stripe, Twilio, SendGrid) maintain backward-compatible APIs
- Internet connectivity is available for all tenant integrations
- Tenants accept shared infrastructure model unless Enterprise plan is purchased

10. Appendices

10.1 Use Case Summary Matrix

UC ID	Name	Actor	Priority
UC-01	Developer Registration and Project Setup	Tenant Developer	High
UC-02	Subscribe to Service Module	Tenant Developer/Admin	High
UC-03	Configure Booking Parameters	Tenant Developer	High
UC-04	End User Creates a Booking	End User (indirect)	High
UC-05	Cancel Booking with Refund	End User (indirect)	High
UC-06	Configure Payment Gateway	Tenant Developer	High
UC-07	Process One-Time Payment	End User (indirect)	High
UC-08	Enqueue Customer	End User (indirect)	Medium
UC-09	Serve Next in Queue	Tenant App	Medium
UC-10	View Config History and Rollback	Tenant Admin	Medium
UC-11	Monitor Platform Health	Tenant Developer/Admin	Medium
UC-12	Manage Webhook Endpoints	Tenant Developer	High

10.2 Module-to-Event Mapping

Module	Events Published	Events Consumed
Booking	booking.created, .confirmed, .cancelled, .reminded, .completed, .no_show	payment.succeeded, payment.failed, config.updated
Payment	payment.initiated, .succeeded, .failed, .refunded, subscription.created, .cancelled, .renewed	booking.created, booking.cancelled, config.updated
Queue	queue.item_added, .item_served, .item_expired, .item_cancelled, .position_updated	config.updated
Notification	notification.sent, .delivered, .failed, .bounced	booking.*, payment.*, queue.*, config.updated
Logging	alert.triggered	All events (subscriber to all topics)
Config Engine	config.updated, config.rollback	None (producer only)

10.3 Configuration Parameter Summary

Module	Parameter	Type	Default	Plan Limit
Booking	slot_duration	integer (min)	30	5-480
Booking	max_bookings_per_day	integer	100	Plan-based
Booking	advance_booking_days	integer	30	1-365
Booking	cancellation_hours	integer	24	0-720
Booking	overbooking_pct	integer (%)	0	0-50
Payment	gateway	enum	stripe	All gateways
Payment	currency	string (ISO)	USD	Any
Payment	retry_attempts	integer	3	0-10
Queue	discipline	enum	FIFO	All types
Queue	max_queue_size	integer	500	Plan-based
Queue	expiry_minutes	integer	30	1-1440
Logging	retention_days	integer	90	Plan-based
Logging	log_level	enum	INFO	All levels